

# *ECE391*

# *Computer System Engineering*

## *Lecture 13*

---

**Prof. Kirill Levchenko and Dong Kai Wang**

Slides Adapted from Prof. Kalbarczyk

University of Illinois at Urbana- Champaign

Spring 2024

# *Announcements*

---

- **MP2** due next Monday March 4.
- **MP3** teams formed by March 5.
  - Signup form (see Piazza).

## Quick Recap: i8259.c

---

- (True or False) EOI is sent during acknowledge.
- (True or False) An interrupt line is *masked* at the start of interrupt handling and *unmasked* at the end.

# *Lecture Topics*

---

- **Linux Interrupt System**
  - Data Structures
  - Handler Installation & Removal
  - Invocation
  - Execution
- **Interrupt Chaining and Tasklets**
- **Long-Term Takeaways:**
  - How an operating system handles interrupts, Linux as case study.
    - Data structures, functions, management.
  - How to understand complex code bases, learn by yourself.

# *General Interrupt Abstractions: Interrupt Chaining*

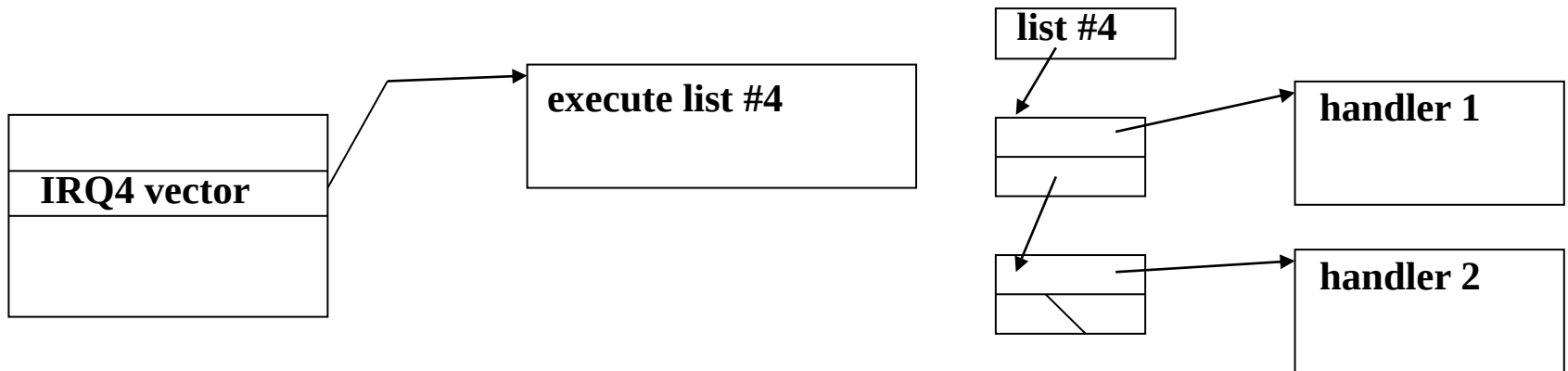
---

- Hardware view: 1 interrupt  $\rightarrow$  1 handler.
- Problems:
  - may have  $> 15$  devices.
  - $> 1$  software routines may want to act in response to device.
  - examples:
    - hotkeys for various functions.
    - move mouse to lower-right corner to start screen-saver.

# General Interrupt Abstractions

## Interrupt Chaining (cont.)

- Solution:
  - interrupt chaining with linked list data structure.
  - (not list embedded into code!)



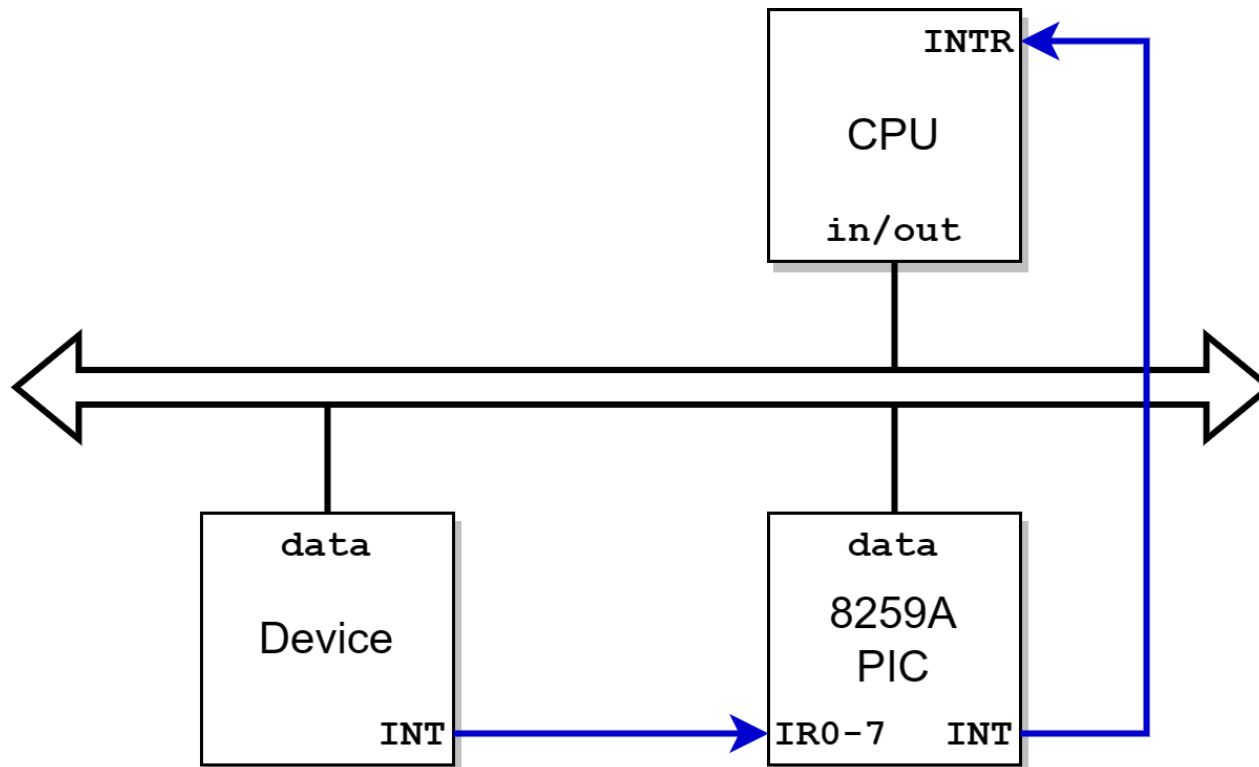
# *General Interrupt Abstractions: Interrupt Chaining (cont.)*

---

- Drawbacks of chaining:
  - for  $> 1$  device
    - must query devices to see if they raised interrupt.
    - not always possible.
  - for 1 device
    - must avoid stealing data/confusing device.
    - Example:
      - by sending two characters to serial port.
      - in response to interrupt declaring port ready for one char.

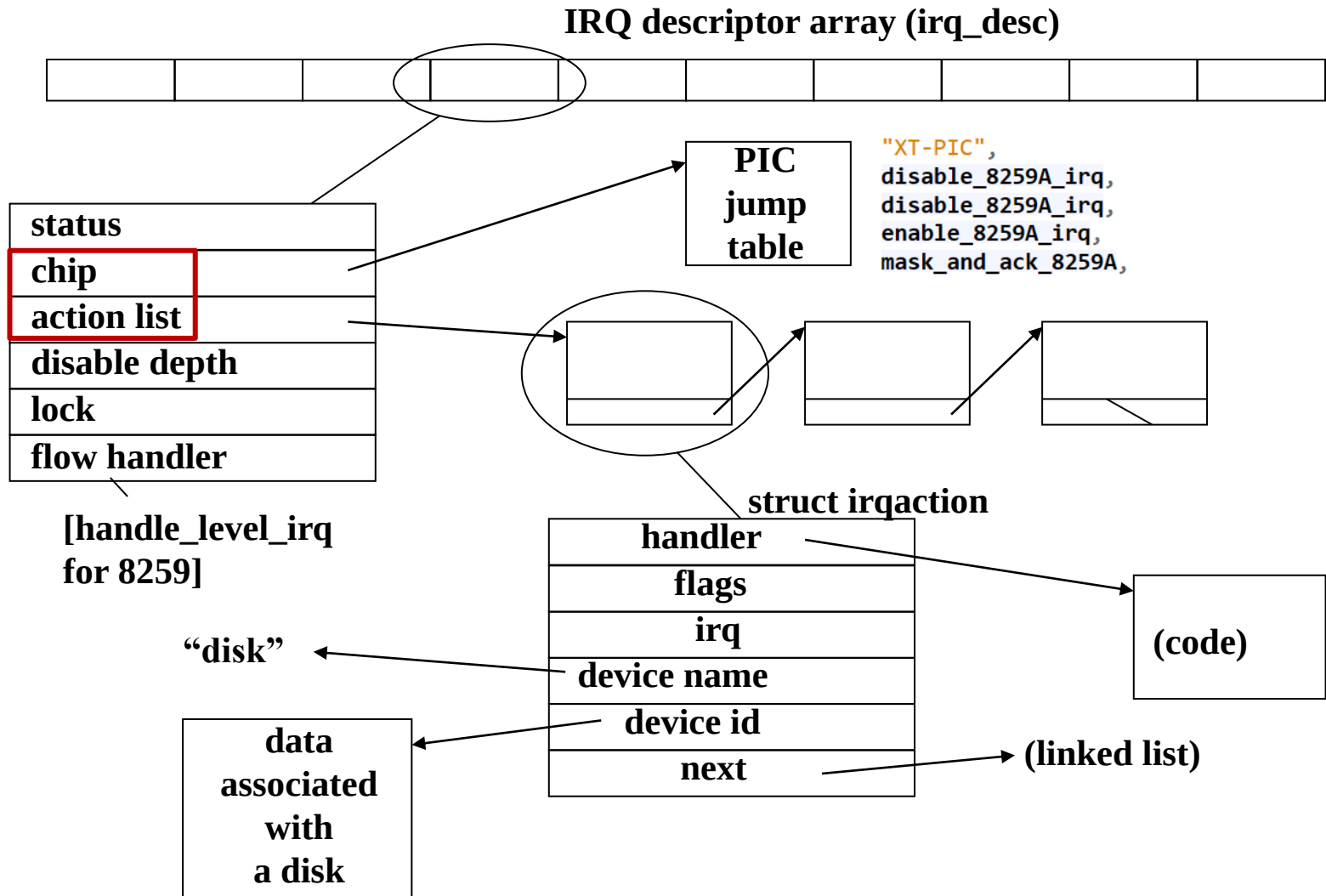
# *Hardware Devices and PIC on System Bus*

- Up to now, we've been mostly talking about OS interaction with PIC, not the device itself.





# Linux' Interrupt Data Structures



# Linux' request\_irq

(kernel/irq/manage.c)

- Linux Source
- We won't have time to cover line by line.

```
/**
 * request_irq - allocate an interrupt line
 * @irq: Interrupt line to allocate
 * @handler: Function to be called when the IRQ occurs
 * @irqflags: Interrupt type flags
 * @devname: An ascii name for the claiming device
 * @dev_id: A cookie passed back to the handler function
 *
 * This call allocates interrupt resources and enables the
 * interrupt line and IRQ handling. From the point this
 * call is made your handler function may be invoked. Since
 * your handler function must clear any interrupt the board
 * raises, you must take care both to initialise your hardware
 * and to set up the interrupt handler in the right order.
 *
 * Dev_id must be globally unique. Normally the address of the
 * device data structure is used as the cookie. Since the handler
 * receives this value it makes sense to use it.
 *
 * If your interrupt is shared you must pass a non NULL dev_id
 * as this is required when freeing the interrupt.
 *
 * Flags:
 *
 * IRQF_SHARED           Interrupt is shared
 * IRQF_DISABLED         Disable local interrupts while processing
 * IRQF_SAMPLE_RANDOM    The interrupt can be used for entropy
 */
```

# *Linux'* request\_irq

```
int  
request_irq (unsigned int irq,  
            irq_handler_t handler,  
            unsigned long irqflags,  
            const char* devname,  
            void* dev_id)
```

← irq # (0-15 for PIC)

← pointer to interrupt handler

← bit vector

← human-readable device name

← pointer to device-specific data

(kernel/irq/manage.c)

```
{  
    struct irqaction* action;  
    int retval;  
  
    if (irq >= NR_IRQS)  
        return -EINVAL;  
    if (!handler)  
        return -EINVAL;
```

```
    action = kmalloc (sizeof (struct irqaction), GFP_ATOMIC);  
    if (!action)  
        return -ENOMEM;
```

← Dynamically allocate  
new action structure for  
linked list of actions

```
    action->handler = handler;  
    action->flags    = irqflags;  
    cpus_clear (action->mask);  
    action->name     = devname;  
    action->next     = NULL;  
    action->dev_id   = dev_id;
```

|             |
|-------------|
| handler     |
| flags       |
| irq         |
| device name |
| device id   |
| next        |

Fill in the new  
action structure

```
    select_smp_affinity (irq);
```

```
    retval = setup_irq (irq, action);
```

← Add the new action structure  
to the link list of actions

```
    if (retval)  
        kfree (action);
```

← Upon failure free the action structure

```
    return retval;  
}
```

# *Comments on `request_irq` – Arguments*

- **`irq`** – irq # (0-15 for PIC).
- **`handler`** – pointer to interrupt handler with following arguments.
  - irq #
  - `dev_id` pointer (type `void*`).
  - handlers should return 1 if handled, 0 if not.
- **`irqflags`** – bit vector.
  - `IRQF_SHARED` – interrupt chaining is allowed.
  - `IRQF_DISABLED` – disable interrupts (IF=0) when handling.
  - `IRQF_SAMPLE_RANDOM` – use device timing as source of random numbers.
- **`devname`** – human-readable device name (visible in `/proc/interrupts`).
- **`dev_id`** – pointer to device-specific data (returned to handler when called).

# Who calls *request\_irq*

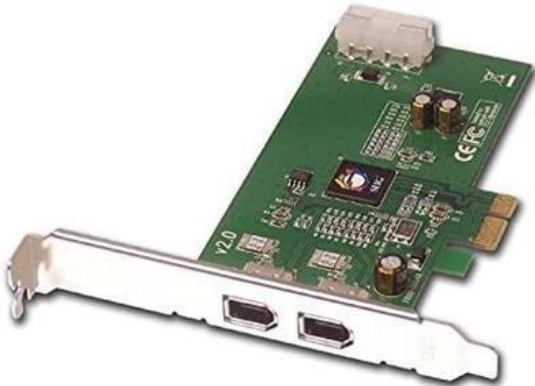
- What is this?



- Who calls the *request\_irq* function?
  - *Drivers*

# *Files Calling request\_irq*

- Calls of *request\_irq* from different files.



drivers/i2c/busses/i2c-pxa.c, line 903  
drivers/i2c/busses/i2c-s3c2410.c, line 829  
drivers/i2c/chips/isp1301\_omap.c, 2 times  
drivers/i2c/chips/tps65010.c, line 554  
drivers/ide/ide-probe.c, line 1114  
drivers/ide/legacy/hd.c, line 809  
drivers/ide/legacy/macide.c, line 132  
drivers/ieee1394/ohci1394.c, line 3405  
drivers/ieee1394/pcilynx.c, line 1254  
drivers/infiniband/hw/amso1100/c2.c, line 1076  
drivers/infiniband/hw/ipath/ipath\_driver.c, line 478  
drivers/infiniband/hw/mthca/mthca\_eq.c, 2 times  
drivers/input/joystick/amijoy.c, line 89  
drivers/input/keyboard/amikbd.c, line 234  
drivers/input/keyboard/corgikbd.c, line 350  
drivers/input/keyboard/gpio\_keys.c, line 79



# Example: Logitech Bus Mouse Handler

```
if (request_irq(logibm_irq, logibm_interrupt, 0, "logibm", NULL)) {  
    printk(KERN_ERR "logibm.c: Can't allocate irq %d\n", logibm_irq);  
    return -EBUSY;  
}  
outb(LOGIBM_ENABLE_IRQ, LOGIBM_CONTROL_PORT);  
return 0;  
  
int  
request_irq (unsigned int irq,  
             irq_handler_t handler,  
             unsigned long irqflags,  
             const char* devname,  
             void* dev_id)  
{  
    static irqreturn_t logibm_interrupt(int irq, void *dev_id)  
    {  
        char dx, dy;  
        unsigned char buttons;  
  
        outb(LOGIBM_READ_X_LOW, LOGIBM_CONTROL_PORT);  
        dx = (inb(LOGIBM_DATA_PORT) & 0xf);  
        outb(LOGIBM_READ_X_HIGH, LOGIBM_CONTROL_PORT);  
        dx |= (inb(LOGIBM_DATA_PORT) & 0xf) << 4;  
        outb(LOGIBM_READ_Y_LOW, LOGIBM_CONTROL_PORT);  
        dy = (inb(LOGIBM_DATA_PORT) & 0xf);  
        outb(LOGIBM_READ_Y_HIGH, LOGIBM_CONTROL_PORT);  
        buttons = inb(LOGIBM_DATA_PORT);  
        dy |= (buttons & 0xf) << 4;  
        buttons = ~buttons >> 5;  
  
        input_report_rel(logibm_dev, REL_X, dx);  
        input_report_rel(logibm_dev, REL_Y, dy);  
        input_report_key(logibm_dev, BTN_RIGHT, buttons & 1);  
        input_report_key(logibm_dev, BTN_MIDDLE, buttons & 2);  
        input_report_key(logibm_dev, BTN_LEFT, buttons & 4);  
        input_sync(logibm_dev);  
  
        outb(LOGIBM_ENABLE_IRQ, LOGIBM_CONTROL_PORT);  
        return IRQ_HANDLED;  
    }  
}
```

## Comments on request\_irq (cont.)

- Start by checking values.  
(two checks from the shown code)

```
if (irq >= NR_IRQS)
    return -EINVAL;

if (!handler)
    return -EINVAL;
```

- Dynamically allocate new action structure for linked list of actions.

```
action = kmalloc(sizeof(struct irqaction), GFP_ATOMIC);
if (!action)
    return -ENOMEM;
```

- Fill in the new action structure.
- Try to add it using **setup\_irq**
  - if call fails, free the action structure.

```
action->handler = handler;
action->flags = irqflags;
cpus_clear(action->mask);
action->name = devname;
action->next = NULL;
action->dev_id = dev_id;
```

```
select_smp_affinity(irq);
```

```
retval = setup_irq(irq, action);
if (retval)
    kfree(action);
```

```
return retval;
```



```

int
setup_irq (unsigned int irq, struct irqaction* new)
{
    struct irq_desc* desc = irq_desc + irq;
    struct irqaction* old;
    struct irqaction** p;
    unsigned long flags;
    int shared = 0;

    if (irq >= NR_IRQS)
        return -EINVAL;
    if (desc->chip == &no_irq_chip)
        return -ENOSYS;

    if (new->flags & IRQF_SAMPLE_RANDOM)
        rand_initialize_irq (irq);

    spin_lock_irqsave (&desc->lock, flags);
    p = &desc->action;
    if ((old = *p) != NULL) {;
        /* Can't share interrupts unless both agree to and are same type. */
        if (((old->flags & new->flags) & IRQF_SHARED) ||
            ((old->flags ^ new->flags) & IRQF_TRIGGER_MASK)) {
            spin_unlock_irqrestore (&desc->lock, flags);
            return -EBUSY;
        }

        /* add new interrupt at end of irq queue */
        do {
            p = &old->next;
            old = *p;
        } while (old);
        shared = 1;
    }

    *p = new;

    if (!shared) {
        irq_chip_set_defaults (desc->chip);
        desc->status &= ~(IRQ_AUTODETECT | IRQ_WAITING | IRQ_INPROGRESS);
        if (!(desc->status & IRQ_NOAUTOEN)) {
            desc->depth = 0;
            desc->status &= ~IRQ_DISABLED;
            desc->chip->startup (irq);
        } else {
            /* Undo nested disables: */
            desc->depth = 1;
        }
    }

    spin_unlock_irqrestore (&desc->lock, flags);

    new->irq = irq;
    register_irq_proc (irq);
    new->dir = NULL;
    register_handler_proc (irq, new);

    return 0;
}

```

# Linux setup\_irq

(kernel/irq/manage.c)

Critical section begins

New action is added at the end of the link list

Interrupt  
descriptor

|               |
|---------------|
| status        |
| chip          |
| action list   |
| disable depth |
| lock          |
| flow handler  |

PIC  
jump table

Startup PIC for this interrupt

Critical section ends

Create new /proc/irq/<irq#> directory

Add handler subdirectory in /proc/irq/<irq#>/<action name>

|                       |
|-----------------------|
| human-readable name   |
| startup function      |
| shutdown function     |
| enable function       |
| disable function      |
| mask function         |
| mask_ack function     |
| unmask function       |
| (+ several others...) |

## *Comments on setup\_irq*

---

- Start with a couple of sanity checks.
- If random sampling flag is set:
  - initialize as source of random numbers.
  - good random numbers are hard to find!
  - devices such as disks can be used because of “random” rotational latency to read data (for example).

## Comments on `setup_irq` (cont.)

- Critical section (most of function)
  - blocks other activity on descriptor
  - uses `irqsave/restore` to allow handlers to be added from any context
- Note that new action goes at the end of the linked list, not the start.
- Interrupt chaining only allowed:
  - if all handlers agree to it.
  - otherwise only first handler is successfully added.

```
struct irqaction {  
    irq_handler_t handler;  
    unsigned long flags;  
    cpumask_t mask;  
    const char *name;  
    void *dev_id;  
    struct irqaction *next;  
    int irq;  
    struct proc_dir_entry *dir;  
};
```

## Comments on `setup_irq` (cont.)

- Go to the end of action linked list and insert.

```
spin_lock_irqsave(&desc->lock, flags);
p = &desc->action;
if ((old = *p) != NULL) {
    /* Can't share interrupts unless both agree to */
    if (!(old->flags & new->flags & SA_SHIRQ)) {
        spin_unlock_irqrestore(&desc->lock, flags);
        return -EBUSY;
    }

    /* add new interrupt at end of irq queue */
    do {
        p = &old->next;
        old = *p;
    } while (old);
    shared = 1;
    struct irq_desc *desc = irq_desc + irq;
    struct irqaction *old, **p;
}
```

```
*p = new;
```

## *Comments on `setup_irq` (cont.)*

---

- If this handler is the first:
  - make sure that PIC jump table has proper default functions.
  - clear some status flags.
  - clear any previous software disablement (depth).
  - call the PIC startup function.
- After critical section:
  - create `/proc/irq/<irq#>` directory if necessary.
  - add handler subdirectory in `/proc/irq/<irq #>/<action name>`.

# Linux `free_irq`

(`kernel/irq/manage.c`)

```
void
free_irq (unsigned int irq, void* dev_id)
{
    struct irq_desc* desc;
    struct irqaction** p;
    unsigned long flags;

    if (irq >= NR_IRQS)
        return;

    desc = irq_desc + irq;
    spin_lock_irqsave (&desc->lock, flags);
    p = &desc->action;
    for (;;) {
        struct irqaction* action = *p;

        if (action) {
            struct irqaction** pp = p;

            p = &action->next;
            if (action->dev_id != dev_id)
                continue;

            /* Found it - now remove it from the list of entries */
            *pp = action->next;

            if (!desc->action) {
                desc->status |= IRQ_DISABLED;
                desc->chip->shutdown (irq);
            }
            spin_unlock_irqrestore (&desc->lock, flags);

            /* Remove from /proc/irq/<irq #> */
            unregister_handler_proc (irq, action);

            /* Make sure it's not being used on another CPU */
            synchronize_irq (irq);

            kfree (action);

            return;
        }
        printk (KERN_ERR "Trying to free already-free IRQ %d\n", irq);
        spin_unlock_irqrestore (&desc->lock, flags);
        return;
    }
}
```

Find the correct irq descriptor

Critical section begins

Search for action in list

The "continue" causes the rest of the statement body in the loop to be skipped.

No more handlers for the IRQ;  
you can invoke PIC shutdown

Critical section ends

Remove handler subdirectory in `/proc/irq/<irq#>/<action name>`

Free the action structure

Critical section ends

## *Comments on free\_irq*

---

- Arguments:
  - `irq` – irq # (0-15 for PIC).
  - `dev_id` – MUST match pointer value used in `request_irq` call.
- Start by checking arguments.
- Critical section starts to prevent manipulations of descriptor.
  - blocks interrupt from starting to execute.
  - (interrupt may already be executing on another processor).

## Comments on *free\_irq* (cont.)

- Search for action in list:
  - based on `dev_id` pointer.
  - hence need for matching pointer.
  - passing NULL `dev_id` not allowed with chained interrupts.
- Once found: (notice that most of the loop executes exactly once)
  - remove the action. (*how are next pointers assigned?*)
  - if this action/handler was the last for this interrupt,
    - turn on software disablement.
    - (doing so signals interrupts waiting for descriptor lock to abort once they obtain the lock)
    - call the PIC shutdown function.

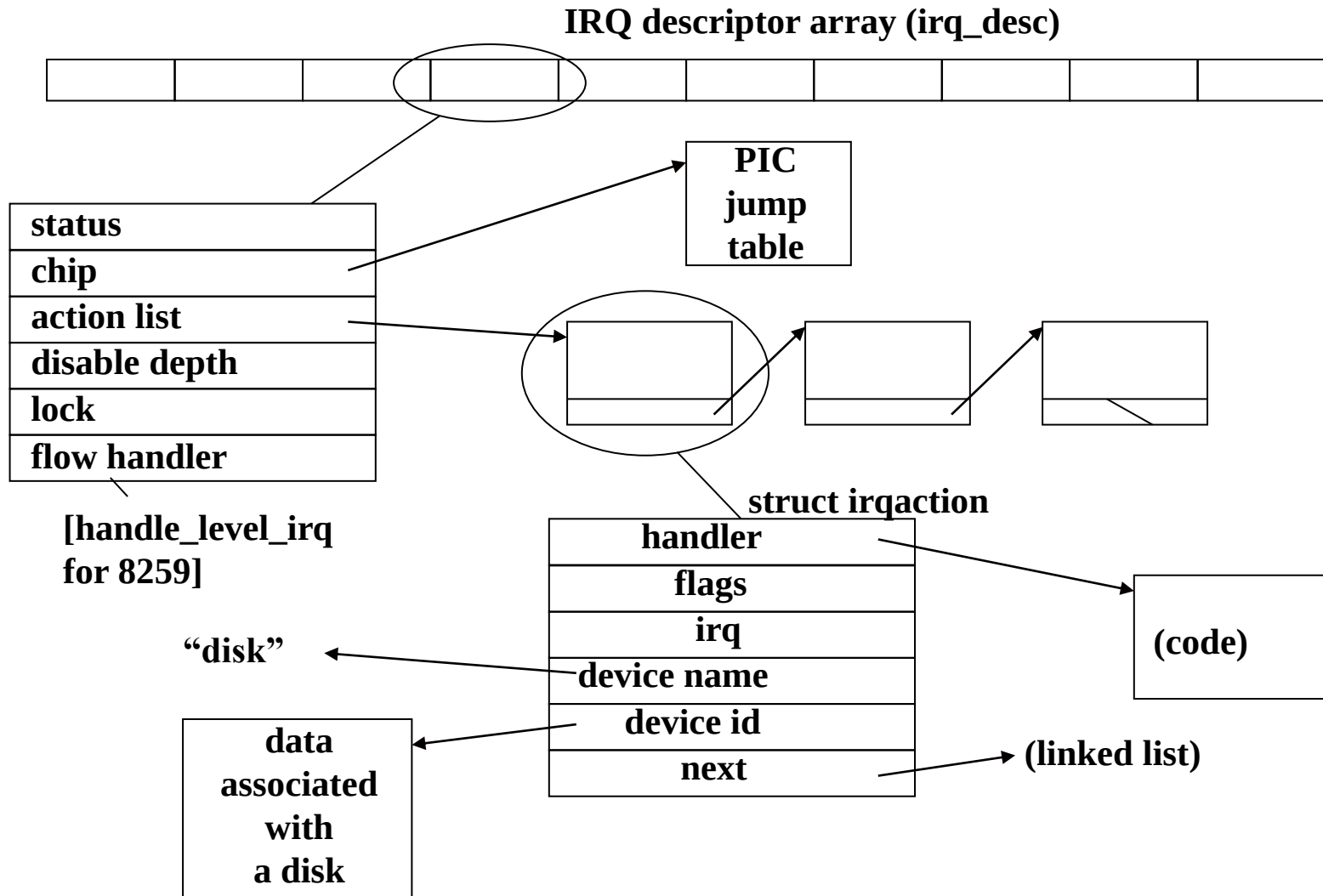


## *Comments on `free_irq` (cont.)*

---

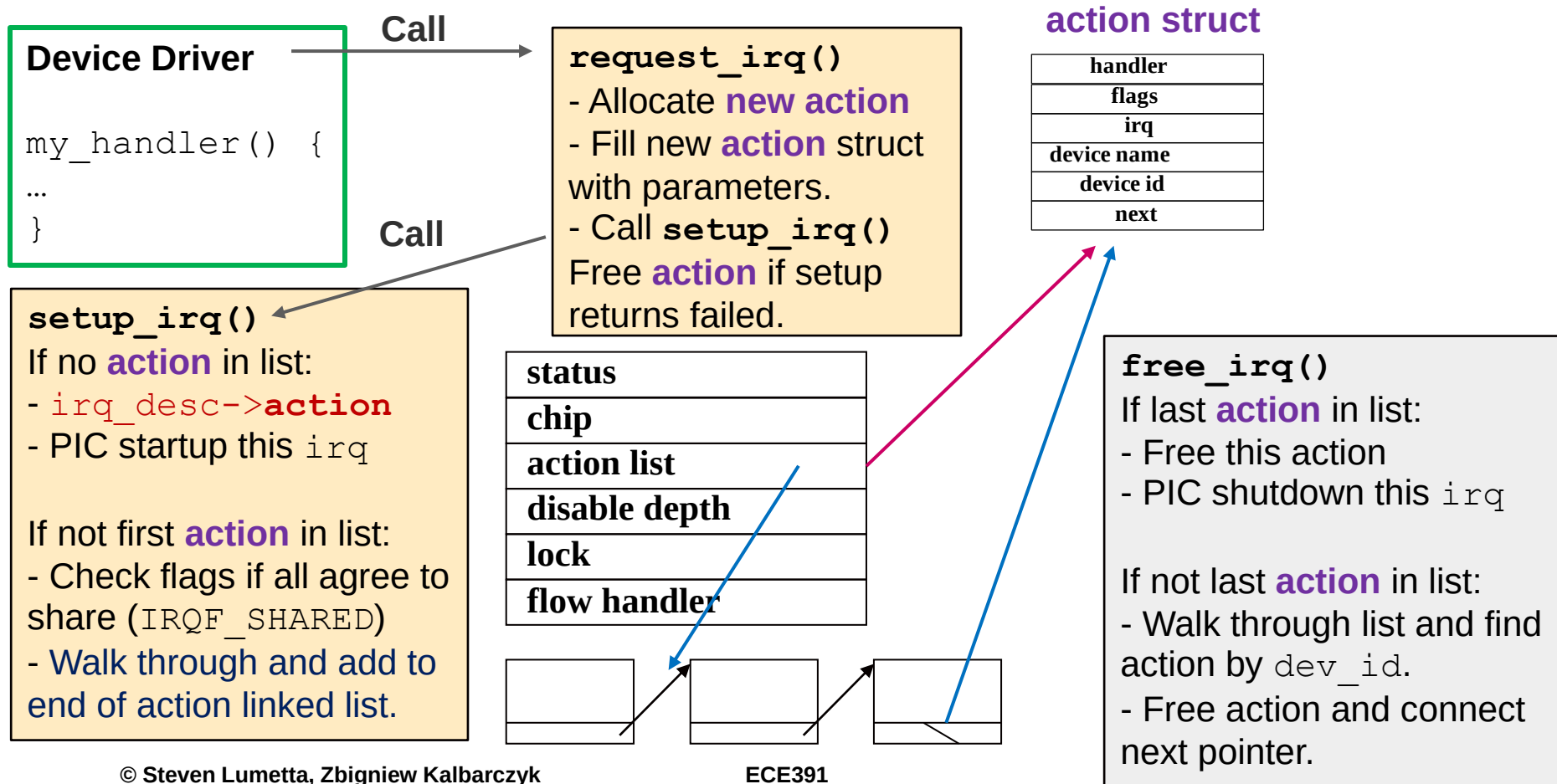
- Remove the `/proc/irq/<irq #>/<action name>` entry.
- On an SMP:
  - interrupt may be executing on another processor.
  - need to wait for it to finish after releasing descriptor lock.
- Finally, free the action structure.

# Linux' Interrupt Data Structures



# Recap: Installing and Freeing Handlers

**Prior to this:** `irq_desc` array is initialized (but no actions), 8259A initialized (sent ICW1-4). Devices are attached to PIC but are all masked, since they don't have interrupt handlers yet.



# Device ID Example

- Device ID pointer (`void* dev_id`) points to device specific data.
  - Used differently by different devices.
  - Cannot be `NULL` if sharing handlers (this pointer used as identifier to remove specific handlers in action list).

```
static irqreturn_t ohci_irq_handler(int irq, void *dev_id)
{
    quadlet_t event, node_id;
    struct ti_ohci *ohci = (struct ti_ohci *)dev_id;
    struct hpsb_host *host = ohci->host;
    int phyid = -1, isroot = 0;
    unsigned long flags;

    struct ti_ohci {
        struct pci_dev *dev;

        enum {
            OHCI_INIT_ALLOC_HOST,
            OHCI_INIT_HAVE_MEM_REGION,
            OHCI_INIT_HAVE_IOMAPPING,
            OHCI_INIT_HAVE_CONFIG_ROM_BUFFER,
            OHCI_INIT_HAVE_SELFID_BUFFER,
        };
    };
}
```



# Interrupt Action Flags

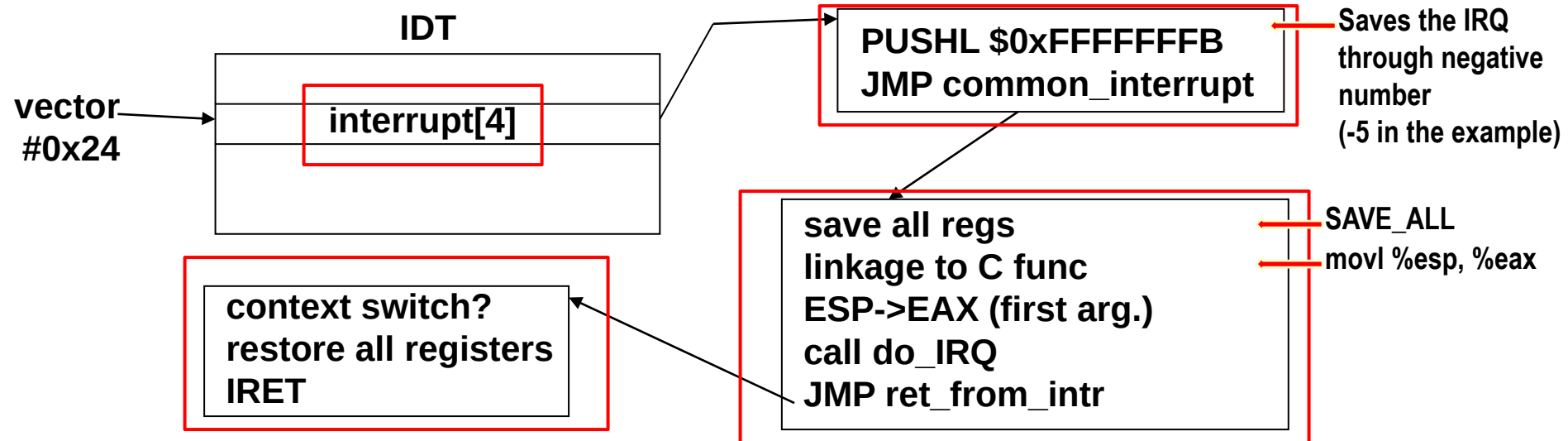
```
* IRQF_DISABLED - keep irqs disabled when calling the action handler
* IRQF_SAMPLE_RANDOM - irq is used to feed the random generator
* IRQF_SHARED - allow sharing the irq among several devices
* IRQF_PROBE_SHARED - set by callers when they expect sharing mismatches to occur
* IRQF_TIMER - Flag to mark this interrupt as timer interrupt
* IRQF_PERCPU - Interrupt is per cpu
* IRQF_NOBALANCING - Flag to exclude this interrupt from irq balancing
*/
#define IRQF_DISABLED          0x00000020
#define IRQF_SAMPLE_RANDOM    0x00000040
#define IRQF_SHARED            0x00000080
#define IRQF_PROBE_SHARED     0x00000100
#define IRQF_TIMER             0x00000200
#define IRQF_PERCPU            0x00000400
#define IRQF_NOBALANCING      0x00000800

/*
 * Migration helpers. Scheduled for removal in 1/2007
 * Do not use for new code !
 */
#define SA_INTERRUPT           IRQF_DISABLED
#define SA_SAMPLE_RANDOM      IRQF_SAMPLE_RANDOM
#define SA_SHIRQ               IRQF_SHARED
#define SA_PROBEIRQ           IRQF_PROBE_SHARED
#define SA_PERCPU              IRQF_PERCPU
```

**Why encoded this way?**

**What if flag parameter is zero?**

# Interrupt Invocation



```
vector=0
ENTRY(irq_entries_start)
.rept NR_IRQS
    ALIGN
1:    pushl $vector-256
    jmp common_interrupt

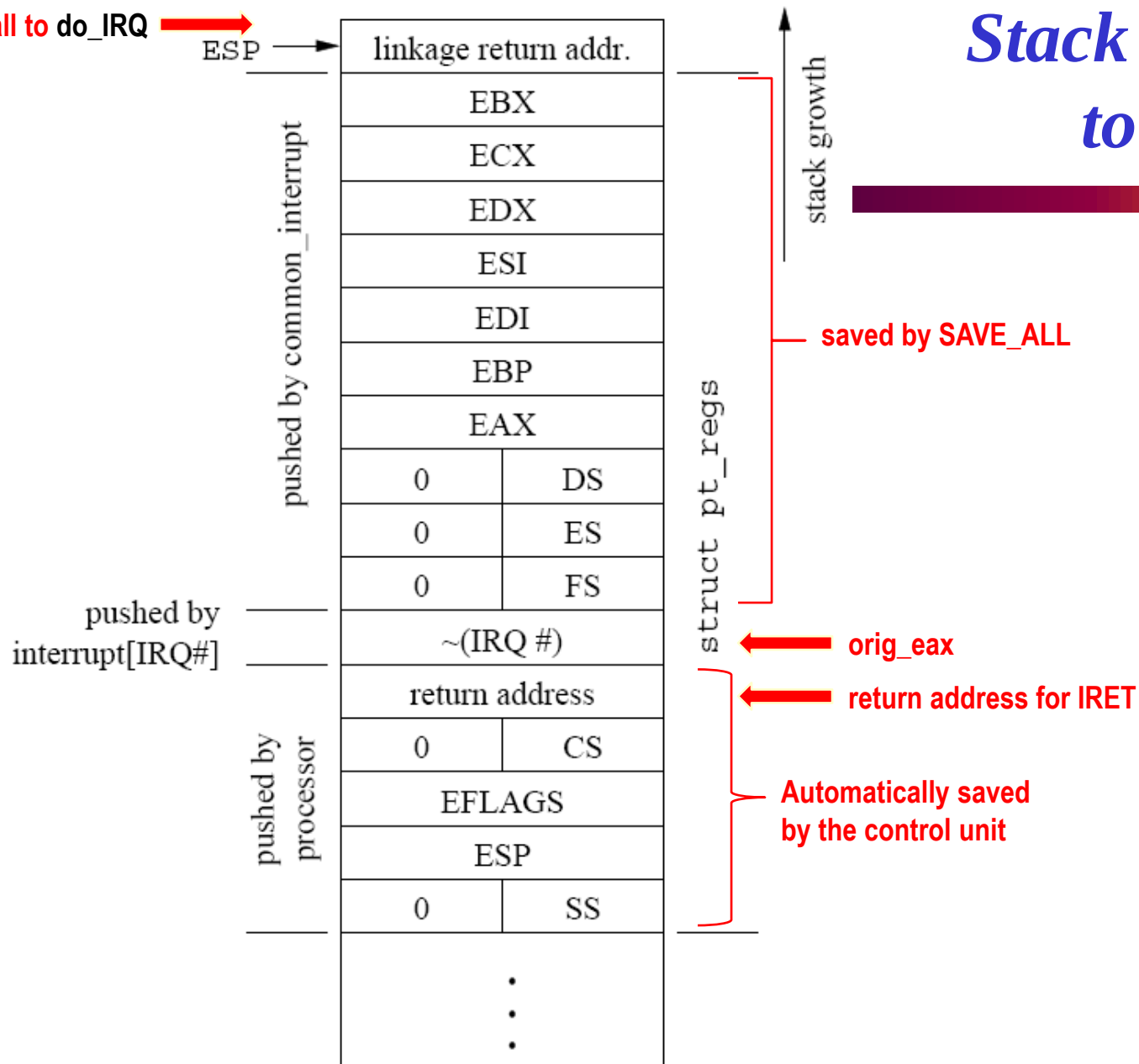
.data
    .long 1b

.text
vector=vector+1
.endr
```

```
common_interrupt:
    SAVE_ALL
    movl %esp,%eax
    call do_IRQ
    jmp ret_from_intr
```

```
#define SAVE_ALL \
    cld; \
    pushl %es; \
    pushl %ds; \
    pushl %eax; \
    pushl %ebp; \
    pushl %edi; \
    pushl %esi; \
    pushl %edx; \
    pushl %ecx; \
    pushl %ebx; \
```

After call to `do_IRQ` →



## Stack on Entry to `do_IRQ`

# *struct pt\_regs*

- Note: `orig_eax` contains `irq` vector (from `pushl` instruction)
- `%eax` actually points to starting address of this struct on call (`fastcall`).

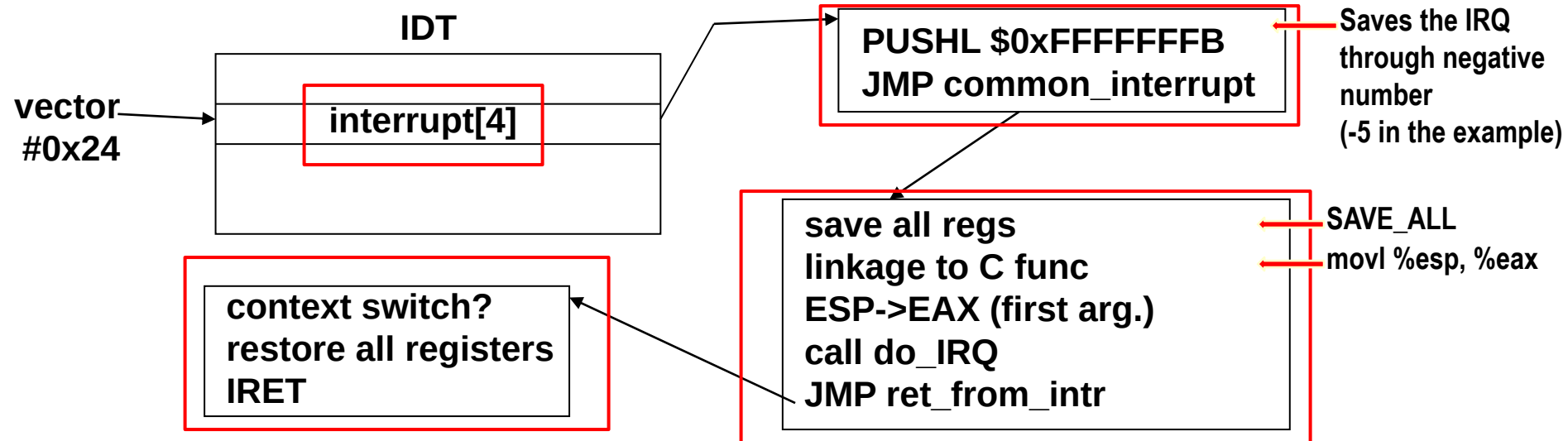
```
vector=0
ENTRY(irq_entries_start)
.rept NR_IRQS
    ALIGN
1:    pushl $vector-256
    jmp common_interrupt
.data
    .long 1b
.text
vector=vector+1
.endr

common_interrupt:
    SAVE_ALL
    movl %esp,%eax
    call do_IRQ
    jmp ret_from_intr

struct pt_regs {
    long ebx;
    long ecx;
    long edx;
    long esi;
    long edi;
    long ebp;
    long eax;
    int xds;
    int xes;
    long orig_eax;
    long eip;
    int xcs;
    long eflags;
    long esp;
    int xss;
};
```



# Interrupt Invocation



- Why funnel all IRQs into one piece of code instead of producing one function per IRQ?
  - kernel code/size versus speed tradeoff:
    - perhaps no big deal for 16.
    - keep in mind that you're saving a tiny number of instructions.
    - APIC used by most SMPs has 256 vectors.

# Overview: Handling Interrupts (Linux)

|                      |
|----------------------|
| <b>status</b>        |
| <b>chip</b>          |
| <b>action list</b>   |
| <b>disable depth</b> |
| <b>lock</b>          |
| <b>flow handler</b>  |

**do\_irq()**

- Retrieve **irq** number
- Do some bookkeeping
- Call **handle\_irq()** function pointer

**handle\_level\_irq()**

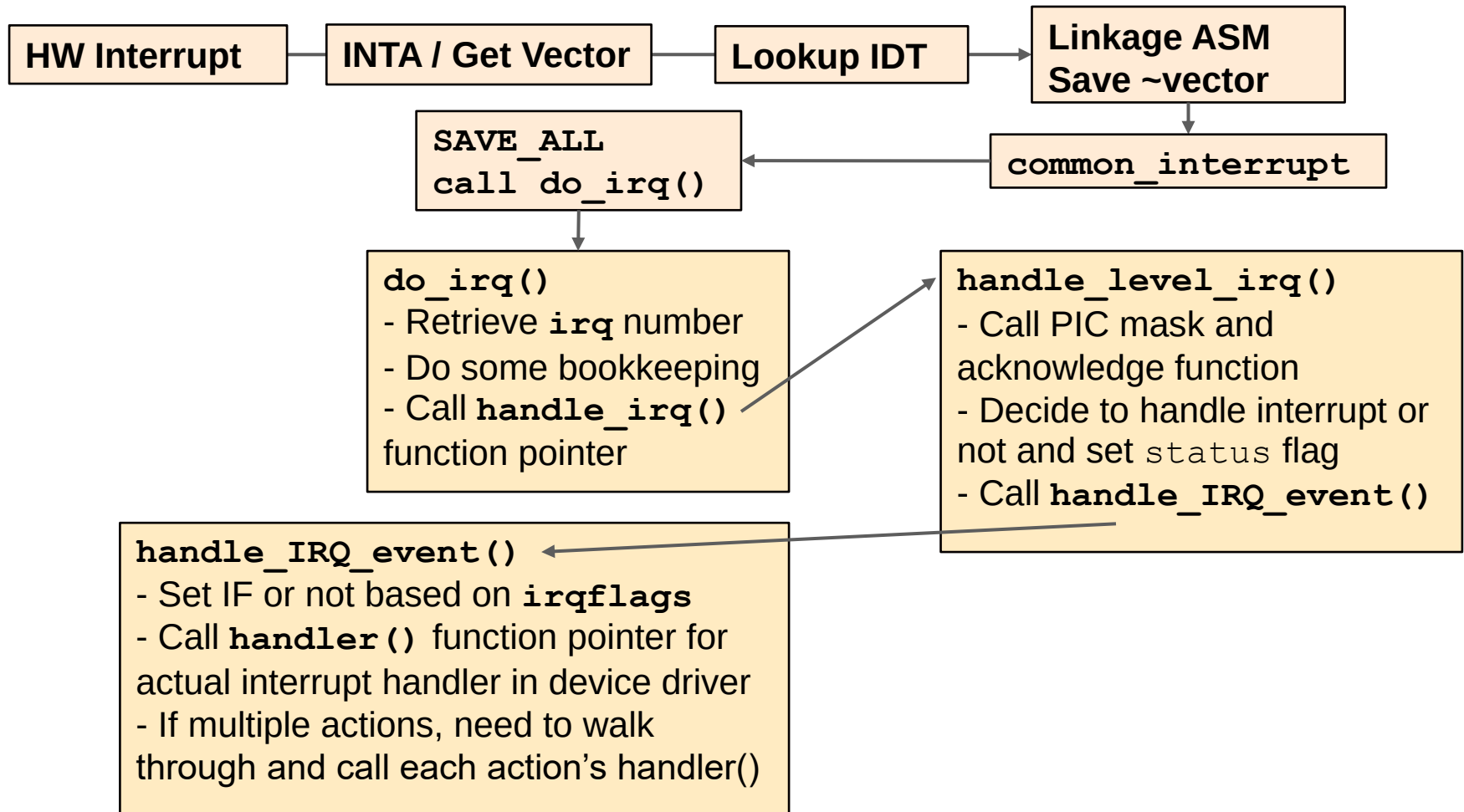
- Call PIC mask and acknowledge function
- Decide to handle interrupt or not and set **status** flag
- Call **handle\_IRQ\_event()**

|                    |
|--------------------|
| <b>handler</b>     |
| <b>flags</b>       |
| <b>irq</b>         |
| <b>device name</b> |
| <b>device id</b>   |
| <b>next</b>        |

**handle\_IRQ\_event()**

- Set IF or not based on **irqflags**
- Call **handler()** function pointer for actual interrupt handler in device driver
- If multiple actions, need to walk through and call each action's **handler()**

# Full Steps: Handling Interrupts (incl. HW)



# Linux do\_IRQ

## arch/i386/kernel/irq.c

```
fastcall unsigned int
do_IRQ (struct pt_regs* regs)
{
    struct pt_regs* old_regs;
    /* high bit used in ret_from_code */
    int irq = ~regs->orig_eax;
    struct irq_desc* desc = irq_desc + irq;

    if ((unsigned)irq >= NR_IRQS) {
        printk (KERN_EMERG "%s: cannot handle IRQ %d\n", __FUNCTION__, irq);
        BUG ();
    }

    old_regs = set_irq_regs (regs);
    irq_enter ();

    /* for 8259A interrupts, handle_irq is set to handle_level_irq */
    desc->handle_irq (irq, desc);

    irq_exit ();
    set_irq_regs (old_regs);
    return 1;
}
```

← fastcall convention used to pass arguments in registers;  
reduces the number of memory accesses required for the call

← gets the interrupt number from the orig\_eax filed in the regs structure

← Record registers at time of interrupt

← Increments counter of nested interrupt handlers

← call interrupt flow handler handle\_level\_irq for all 8259A interrupts

← Exit irq context and process softirqs if needed

## *Interrupt Invocation (cont.)*

- Signature for do\_IRQ:

```
fastcall unsigned int do_IRQ  
    (struct pt_regs* regs) ;
```

- **fastcall** macro in Linux tells gcc to pass args in EAX, EDX, ECX.
- EAX points to saved registers (**regs** argument).
- saved registers already on stack.
- note that processor clears IF when it takes an interrupt.
- EFLAGS stored by processor include original IF value.

## *Comments on do\_irq*

---

- Value pushed by irq-specific code re-converted to find irq #.
- Start with a sanity check.
- **set\_irq\_regs** calls record registers at time of interrupt.
  - per-CPU storage.

## *Comments on do\_irq (cont.)*

---

- Central component:
  - call interrupt flow handler,
  - `handle_level_irq` for all 8259A interrupts.
- Return value ignored. (probably intended to indicate interrupt handled)

```
fastcall void
handle_level_irq (unsigned int irq, struct irq_desc* desc)
{
    unsigned int      cpu = smp_processor_id ();
    struct irqaction* action;
    irqreturn_t        action_ret;
```

# Linux `handle_level_irq`

`kernel/irq/chip.c`

```
spin_lock (&desc->lock);
mask_ack_irq (desc, irq);
```

← Critical section begins

← Call PIC's `mask_ack` function

```
if (desc->status & IRQ_INPROGRESS)
    goto out_unlock;
```

← If interrupt already in progress, do nothing

```
desc->status &= ~(IRQ_REPLAY | IRQ_WAITING);
kstat_cpu (cpu).irqs[irq]++;
```

```
/*
 * If its disabled or no action available
 * keep it masked and get out of here
 */
```

```
action = desc->action;
if (!action || (desc->status & IRQ_DISABLED)) {
    desc->status |= IRQ_PENDING;
    goto out_unlock;
}
```

← Atomically decide whether to execute handlers

```
desc->status |= IRQ_INPROGRESS;
desc->status &= ~IRQ_PENDING;
spin_unlock (&desc->lock);
```

← Set status for execution: in-progress and no longer pending

← Critical section ends

```
action_ret = handle_IRQ_event (irq, action);
if (!noirqdebug)
    note_interrupt (irq, desc, action_ret);
```

← Handler execution done via `handle_IRQ_event`

```
spin_lock (&desc->lock);
desc->status &= ~IRQ_INPROGRESS;
if (!(desc->status & IRQ_DISABLED) && desc->chip->unmask)
    desc->chip->unmask (irq);
```

← When done, remove in-progress flag and unmask on PIC (unless disabled)

```
out_unlock:
```

```
spin_unlock (&desc->lock);
```

← Critical section ends

```
}
```



## *Comments on `handle_level_irq`*

---

- Critical section starts:
  - to read descriptor status and action (handler) list.
  - IF=0 at this point (set by processor when taking interrupt).
- Immediately call PIC's `mask_ack` function (via a wrapper function).
- If interrupt already in progress, do nothing.
  - Interrupt already re-masked and re-acknowledged on PIC.
- Remove software replay.
- Kernel statistics track # of interrupts seen (see `/proc/interrupts`).

## *Comments on `handle_level_irq` (cont.)*

---

- Atomically decide whether to execute handlers immediately:
  - do so if handler defined and not disabled by software.
  - if not, skip to end,
    - mark as pending and end interrupt handling.
    - replayed after last nested `enable_irq` call.
- Set status for execution: in-progress, and no longer pending.

## *Comments on `handle_level_irq` (cont.)*

- Handler execution done via `handle_IRQ_event`
  - done without descriptor lock.
    - allows handlers to use infrastructure.
    - e.g., `enable_irq/disable_irq`, `request_irq/free_irq`
  - usually done with `IF=1` (changed inside `handle_IRQ_event`)
    - done without descriptor lock,
    - Otherwise, this code can deadlock with self.
- When done, remove in-progress flag and unmask on PIC (unless disabled).

# Linux `handle_IRQ_event`

## `kernel/irq/handle.c`

```
irqreturn_t
handle_IRQ_event (unsigned int irq, struct irqaction* action)
{
    irqreturn_t    ret;
    irqreturn_t    retval = IRQ_NONE;
    unsigned int   status = 0;

    /* call specific to ARM processor (to disambiguate timer ticks) */
    handle_dynamic_tick (action);

    if (!(action->flags & IRQF_DISABLED))
        local_irq_enable_in_hardirq ();

    do {
        ret = action->handler (irq, action->dev_id);
        if (ret == IRQ_HANDLED)
            status |= action->flags;
        retval |= ret;
        action = action->next;
    } while (action);

    if (status & IRQF_SAMPLE_RANDOM)
        add_interrupt_randomness (irq);
    local_irq_disable ();

    return retval;
}
```

call translates basically to STI  
(local means "on this CPU")

Walk through list; call each handler  
(return value 1 means handled)

Turn interrupts back off (IF=0 / CLI)

© Steven Lumetta, Zbigniew Kalbarczyk

## *Comments on `handle_IRQ_event`*

---

- Set IF=1 unless the first handler asked to be executed with IF=0.
  - indicated by `IRQF_DISABLED` flag.
  - call translates basically to STI (local means “on this CPU”).
- Walk through list:
  - call each handler,
  - return value 1 means handled.
- Generate random numbers if desired.
- Turn interrupts back off (IF=0 / CLI).

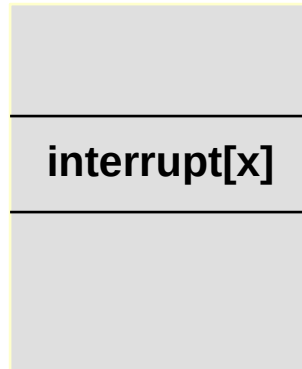
# Summary on Descriptor Flags

- **IRQ\_PENDING** — interrupt raised by hardware; waiting to be executed.
- **IRQ\_INPROGRESS** — some processor is executing handlers.
- **IRQ\_DISABLED** — interrupt disabled in software; postpone execution.
- Many more ...

```
/* Internal flags */
#define IRQ_INPROGRESS      0x00000100
#define IRQ_DISABLED       0x00000200
#define IRQ_PENDING        0x00000400
#define IRQ_REPLAY         0x00000800
#define IRQ_AUTODETECT     0x00001000
#define IRQ_WAITING        0x00002000
#define IRQ_LEVEL          0x00004000
#define IRQ_MASKED         0x00008000
#define IRQ_PER_CPU        0x00010000
#define IRQ_NOPROBE       0x00020000
#define IRQ_NOREQUEST      0x00040000
#define IRQ_NOAUTOEN       0x00080000
#define IRQ_WAKEUP         0x00100000
#define IRQ_MOVE_PENDING   0x00200000
#define IRQ_NO_BALANCING   0x00400000
```

# Interrupt Descriptor Table (IDT)

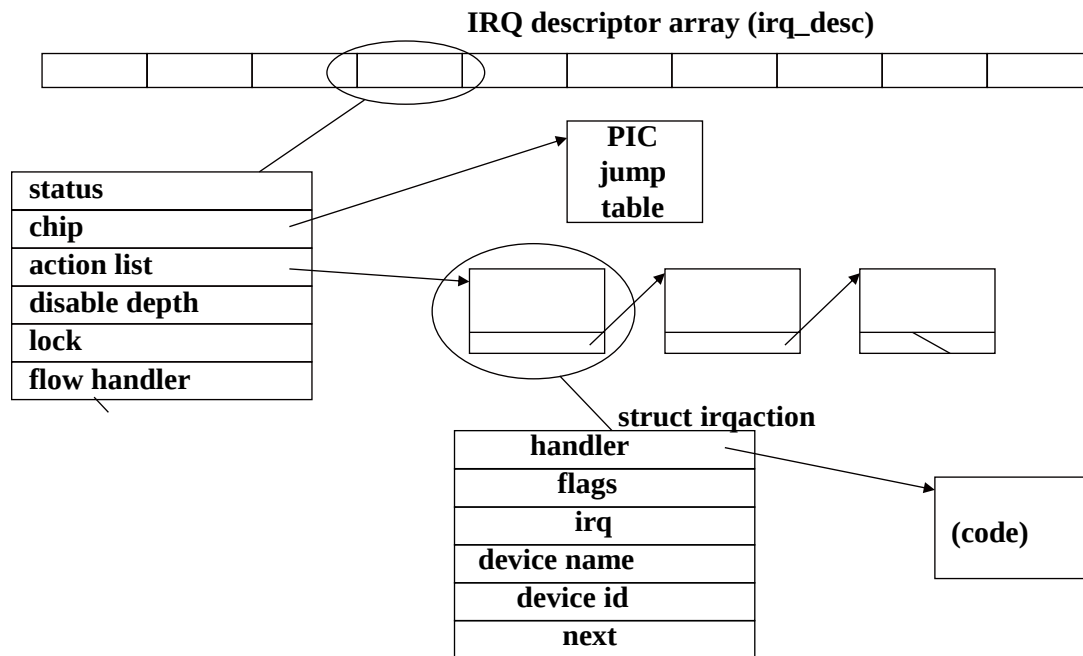
## IDT



- Associates the interrupt line with the int. handler routine.
- 256 entries (each 8-bytes) or descriptors; each corresponds to an interrupt vector.
  - hardware interrupts mapped into vectors 0x20 to 0x2F.
- All Linux interrupt handlers are activated by so called: *interrupt gates* (a descriptor type).
- Whenever an interrupt gate is hit, interrupts are disabled automatically by the processor.

- Before the kernel enables interrupts, it must initialize the *idtr* register to point to the IDT table. (set by the kernel using `lidt` instruction)
- **IDT Initialization**
- Kernel initialization (`setup_idt()`) fills all the 256 entries of IDT with the provisional (or null) handler.

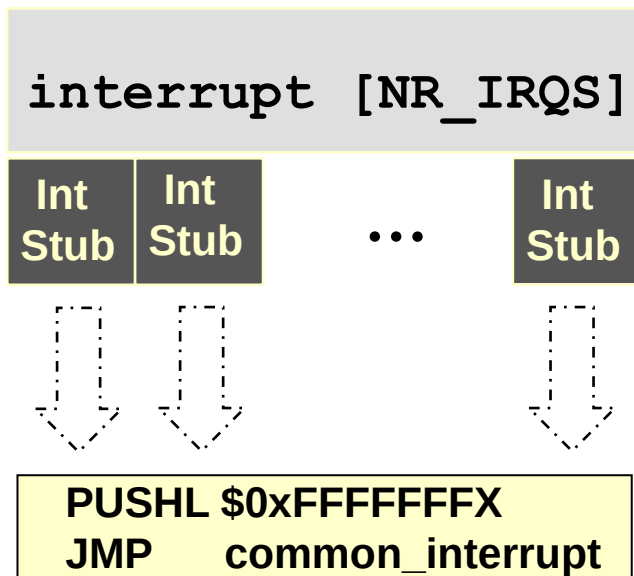
# *irq\_desc* Array



- Every interrupt vector has its own `irq_desc_t` descriptor.
- Descriptors are grouped together in `irq_desc` array, a data structure supported by Linux.
- When a device driver calls the `request_irq()` function a new structure to represent the handler is allocated and initialized.



# *interrupt[NR\_IRQS] Array*



- Kernel maintains one global array of function pointers (`interrupt[NR_IRQS]`) in which it stores pointers to interrupt stubs (`NR_IRQS` is 16 if we use the PIC).

```
vector=0
ENTRY(irq_entries_start)
.rept NR_IRQS
    ALIGN
1:    pushl $vector-256
        jmp common_interrupt
.data
        .long 1b
.text
vector=vector+1
.endr
```

# *Initialization of Interrupt Gates*

- During initialization `init_IRQ()` sets the status field of each IRQ descriptor to **IRQ\_DISABLED**.
- `init_IRQ()` updates the IDT by replacing the provisional interrupt gates with new ones.

```
for (i = 0; i < NR_IRQS; i++)  
    if (i+32 != 128)  
        set_intr_gate(i+32, interrupt[i]);
```

- Interrupt gates are set to the addresses found in the `interrupt[NR_IRQS]` array.